

saperlipopette, a risk-free playground for learning more Git

Maëlle Salmon¹ and Yanina Bellini Saibene¹

1 rOpenSci

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Submitted: 07 May 2026

Published: unpublished

License

Authors of papers retain copyright¹ and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

Git has a challenging learning curve, but it is crucial for both individual and team software development practices to have a good command of a wide set of Git features beyond the basics. We present *saperlipopette*; an open-source, MIT-licensed R package (library) aimed at providing hands-on, low-stakes practice opportunities for Git users to improve their version control skills. We have used this package in several training sessions, online and in person, in English, French and Spanish to teach more advanced Git skills. While *saperlipopette* is best suited for R users, the approach of launching exercises through a single command, creating a (temporary) folder with a Git playground, providing instructions and hints, could be re-used for creating a library or CLI in any language. Even the actual scenarios and guidance could be re-used and ported to another language.

Statement of need

Competent use of version control software improves the software development productivity of individuals and teams by enabling them to work on different features simultaneously while carefully tracking the project history. Improving one's version control skills beyond the basics quickly pays off in time gains, for instance by using dedicated commands to undo changes instead of manually deleting and copy-pasting. It also reduces the risks of losing work as one knows how to avoid mistakes and how to recover from them. Git is the most popular version control tool and furthermore, a huge part of the world's open-source software development happens on GitHub, and to a lesser extent on GitLab and Codeberg, which all use Git. However, Git is notoriously confusing and difficult to learn (Tyagi et al., 2025), which makes upskilling potentially daunting. The difficulty of learning Git is even greater for a non-English speakers.

The target audience of *saperlipopette* consists of professionals developing software who usually work with R and know basic Git commands such as add, commit, push, and pull, and who understand a bit about branches, but are not yet fluent, and therefore underuse Git's palette of features. The package is suitable for instructors to use as a safe practice environment during workshops when live-coding and running hands-on exercises, as well for for self-guided learners.

Design of the software

Maëlle Salmon was inspired to create *saperlipopette* <https://docs.ropensci.org/saperlipopette/> after she read the "Git in Practice" book (McQuaid, 2014) and the "Oh Shit, Git!"¹ website (Sylor-Miller, 2024). She designed this R package as a tool for

¹The package's initial set of functions covered "Oh Shit, Git", which inspired the package name: "*saperlipopette*" is an old-fashioned curse word in French, Maëlle's native language.

locally practicing advanced Git commands, which are not usually taught at beginners workshops and as a way to bridge the gap between knowing about techniques and applying them in day-to-day work. She built saperlipopette on top of gert (Ooms, 2026) and usethis (Wickham et al., 2025). Next, Yanina Bellini Saibene joined the team to help translate exercises and to develop training curricula and workshops, which influenced the documentation and contents of the software. Together Maëlle and Yanina have translated the exercises and training materials for saperlipopette into English, French, and Spanish, and have used it to provide advanced Git training as part of the rOpenSci Champions Program Training.

This package provides learners with a dozen exercises based on active learning pedagogical strategies of working on authentic tasks, with guided practice. Each exercise is focussed on a realistic and useful scenario, such as undoing a change or using Git's time machine to repair a project, and which are each recreated in a small throw-away directory. Exercises are local, meaning users can practice on their machines, with the tools they are already familiar with or are trying to become familiar with: Git on the command line, Git with an interface such as RStudio IDE or Positron IDE, or other general-purpose Git clients. This approach helps reduce the learners' cognitive load as recreating an exercise is only ever an R function call away. Users can practice anytime they want, even getting a refresher on a command before applying it to their real work project as scaffolding strategy.

Our software provides two levels of guidance for each exercise: a general description of the problem to be solved, with the option to request more hints if necessary. Moreover, the messages of the functions, that is to say the exercises and their hints, are available in three languages: English, Spanish and French.

Lastly, from a trainer's perspective, saperlipopette's ready-made exercises can save time in demonstrating Git workflows.

Pedagogical content of the software

Functions, therefore exercises, are organized in three categories in the package's online reference index: "Tools to check your set up", "Exercises inspired from Oh shit Git!", "Clean history", "Use history". The documentation for each exercise states which Git commands it uses. One could type "reset" in the search bar of the [documentation website](#) to find the relevant exercise(s).

Table 1. Overview of the Git scenarios, main concepts and commands covered in *saperlipopette*

Scenario	Main Git concepts	Git commands typically taught
Go back in time	Commit history, HEAD movement, recovering lost states	git reset, git reflog
Fix an accidental commit to main	Branch pointers, rewriting history, moving commits	git reset --hard, git branch, git switch, git checkout
Fix a commit on the wrong branch	Applying commits across branches, history manipulation	git cherry-pick, git reset, git switch, git checkout
Change the last commit message	Amending commits, metadata editing	git commit --amend
Add a small fix to the last commit	Amending content without changing metadata	git commit --amend --no-edit
Undo an older commit	Commit history inspection, safe undo with new commits	git log, git revert

Scenario	Main Git concepts	Git commands typically taught
Discard changes in a file	Restoring file state, working tree vs commit history	git log, git restore, git checkout, git commit
Split changes into multiple commits	Partial staging, atomic commits	git add --patch, git add -p
Remove untracked files	Cleaning the working directory, untracked files	git clean
Experience merge conflicts	Branch merging, conflict detection and resolution	git merge
Clean up commit history	History rewriting, interactive rebase	git rebase -i
Start over from scratch	Resetting the index, unstaging changes	git reset --mixed
Find the commit that introduced a bug	Binary search in commit history	git bisect
Find which commit deleted a file	Using commit history	git log
Find which commit deleted a line	Using commit history	git log -S, git log -G
Understand ancestry references	Ancestry references, commit navigation	git rev-parse, git log, git show
Find who added a specific line	Line-level history, authorship	git blame
See the project at a specific version	Worktrees, tags	git log, git worktree, git tag

In workshops, we select exercises to comply with the learning objectives. We use an informative slidedeck to provide context and explain concepts. We demonstrate exercises using live coding, then have participants solve them. In online workshops, we group participants in breakout rooms to foster collaborations and peer-instruction. After each practice time, and before moving on to the next topic, we discuss participants' questions and remarks about the exercise(s) they have just worked on.

Teaching with saperlipopette

We developed two training curricula which use saperlipopette to demonstrate and provide practice opportunities when learning Git. The first training covers creating a clean Git history using basic and more advanced techniques (Salmon, 2026a, 2026b; Salmon & Bellini Saibene, 2026b); and the second follows the structure of the Oh Shit, Git! Website and covers fixing common Git mishaps (Salmon & Bellini Saibene, 2026a). Both curricula have been used in 1-hour to 2-hours workshops, in person and online, and have been part of conferences, user groups and rOpenSci Champions Program Training. In feedback collected after teaching, students appreciated the hands-on approach and easy set up of saperlipopette exercises.

The multilingual feature improve the effectiveness of saperlipopette in teaching and learning situations. Presenting exercises and feedback in learners' native languages reduces cognitive load and allows them to focus on understanding Git concepts rather than translating technical English. For instructors, multilingual support enables reuse of the same materials across courses, workshops, and regions increasing reach and saving time on preparing exercises and materials.

References

- McQuaid, M. (2014). *Git in practice* (p. 272). Manning Publications. ISBN: 9781617291975
- Ooms, J. (2026). *Gert: Simple git client for r*. <https://docs.ropensci.org/gert/>
- Salmon, M. (2026a). *Git, un gentil 'push' vers une meilleure maîtrise*. Zenodo. <https://doi.org/10.5281/zenodo.18457383>
- Salmon, M. (2026b). *Painlessly improve your git history*. Zenodo. <https://doi.org/10.5281/zenodo.18457425>
- Salmon, M., & Bellini Saibene, Y. N. (2026a). *¡Miércoles, git!* Zenodo. <https://doi.org/10.5281/zenodo.18457460>
- Salmon, M., & Bellini Saibene, Y. N. (2026b). *Usar y mejorar tu historial de git*. Zenodo. <https://doi.org/10.5281/zenodo.18457484>
- Sylor-Miller, K. (2024). *Oh shit, git!?!* <https://ohshitgit.com/>
- Tyagi, D., Kamboj, A., Sharma, A., Sharma, S., & Sharma, R. (2025). Interactive git learning platform: Analysis and implementation of GitMaster playground. *International Journal of Sciences and Innovation Engineering*, 2(6), 181–198. <https://doi.org/10.70849/IJSCI>
- Wickham, H., Bryan, J., Barrett, M., & Teucher, A. (2025). *Usethis: Automate package and project setup*. <https://github.com/r-lib/usethis>